

Домашка 05 — Мережі. Частина 2

Автор: Андрій Пономарьов / Claude Code

Завдання: порахувати для 172.20.0.0/22 адресу мережі, broadcast, діапазон і кількість хостів; нарізати блок на 4 рівні підмережі /24; знайти свою приватну адресу й маску та визначити свою підмережу; перевірити наявність IPv6 і записати висновок. Опціонально: VLSM-план VPC, `conntrack -L`, `dig AAAA + curl -6` проти А-запису.

Примітка: робота виконувалась на macOS, де немає `ip addr` та `conntrack` (обидві — з linux-пакетів `iproute2/netfilter`). Використано нативний еквівалент `ifconfig`, а `conntrack` продемонстровано в Docker Desktop (Linux-ядро 6.12).

1. 172.20.0.0/22 — розрахунок

Маска /22 у двійковому вигляді:

11111111.11111111.11111100.00000000 = 255.255.252.0
└─── 22 біти мережі ─┘ 10 біт ─┘

Межа маски проходить усередині третього октета, крок блоку в ньому $256 - 252 = 4$, тому мережа накриває третій октет від 0 до 3:

Параметр	Значення	Як отримано
Адреса мережі	172.20.0.0	усі 10 хостових біт = 0
Broadcast	172.20.3.255	усі 10 хостових біт = 1
Діапазон хостів	172.20.0.1 – 172.20.3.254	між мережею і broadcast
Кількість хостів	1022	$2^{10} - 2 = 1024 - 2$

«–2» — бо адреса мережі та broadcast хостам не роздаються. Викладки у двійковому вигляді: [docs/subnet-calc.txt](#).

2. Нарізка на 4 × /24

Щоб з /22 отримати 4 підмережі, позичаємо 2 біти з хостової частини ($2^2 = 4$). Кожна /24 — це 256 адрес, з них 254 хостових:

Підмережа	Мережа	Діапазон хостів	Broadcast	Хостів
1	172.20.0.0/24	172.20.0.1 – 172.20.0.254	172.20.0.255	254

Підмережа	Мережа	Діапазон хостів	Broadcast	Хостів
2	172.20.1.0/24	172.20.1.1 – 172.20.1.254	172.20.1.255	254
3	172.20.2.0/24	172.20.2.1 – 172.20.2.254	172.20.2.255	254
4	172.20.3.0/24	172.20.3.1 – 172.20.3.254	172.20.3.255	254

Перевірка: $4 \times 256 = 1024$ адреси — рівно весь блок /22, без дірок і перетинів.

3. Моя приватна адреса і підмережа

```
$ ifconfig en0
    inet 10.20.50.89 netmask 0xfffff00 broadcast 10.20.50.255
```

Що це означає: моя адреса — **10.20.50.89**, маска **0xfffff00** — це 255.255.255.0, тобто **/24**. Отже я в підмережі **10.20.50.0/24**: хости 10.20.50.1–254, broadcast 10.20.50.255, я — хост **.89**. Діапазон приватний (10.0.0.0/8, RFC 1918) — в інтернет такі адреси не маршрутизуються, назовні мене випускає NAT на роутері. Шлюз **10.20.50.1** (відомий ще з ДЗ 4) — у тій самій підмережі, тому доступний напряму по L2, без маршрутизації. Повний вивід: [docs/ifconfig-output.txt](#).

4. Чи є в мене IPv6

macOS-еквівалент `ip -6 addr`:

```
$ ifconfig | grep inet6
    inet6 ::1 prefixlen 128
    inet6 fe80::1445:ef77:ba10:85da%en0 prefixlen 64 secured
    inet6 fd5e:875a:a143:4f74:10a9:6486:9fcd:9da4 prefixlen 64 autoconf
    ... (fe80:: на кожному інтерфейсі)
```

Видно три типи адрес — і жодної глобальної:

- **::1** — loopback;
- **fe80::/10** — link-local, автоматичні, працюють лише в межах сегмента;
- **fd5e:...** — ULA (fd00::/8), приватна IPv6-адреса від роутера, аналог RFC 1918. **Глобальної адреси з 2000::/3 немає.**

Практична перевірка замість test-ipv6.com (сайт джаваскриптовий, тому через його дзеркала):

```
$ curl "https://ipv4.test-ipv6.com/ip/?asn=1"
{"ip":"91.226.253.120","type":"ipv4","asn":"16044","asn_name":"IPORT ... UA"}

$ curl "https://ipv6.test-ipv6.com/ip/"
* Could not resolve host: ipv6.test-ipv6.com
```

```
$ curl -g "https://[2a00:1450:4001:c0f::64]/" # IPv6-адреса Google
* Immediate connect fail: No route to host
```

Висновок: IPv6-інтернету в мене немає. Є тільки link-local і приватна ULA від роутера — провайдер (IPORT, AS16044) глобальний IPv6 не видає. Увесь зовнішній трафік іде по IPv4 через NAT; test-ipv6.com показав би 0/10. Цікава деталь: `curl -6 https://google.com` при цьому «працює», але підключається до `::ffff:142.251.110.100` — це IPv4-mapped адреса, тобто система тихцем підмінила IPv6 на IPv4. Повні виводи: docs/ipv6-check-output.txt.

5. Опціонально: VLSM-план VPC (web / app / db)

Вихідний блок — той самий 172.20.0.0/22 (1022 адреси). Вимоги типового трирівневого застосунку: web-tier наймасштабніший (autoscaling, до ~200 інстансів), app — удвічі менший (~100), db — маленький (~20 реплік). Правило VLSM: нарізаємо від найбільшої підмережі до найменшої, кожна починається на межі, кратній власному розміру, — тоді немає ні дірок, ні перетинів:

Tier	Підмережа	Маска	Діапазон хостів	Broadcast	Хостів	Потреба
web	172.20.0.0/24	255.255.255.0	172.20.0.1 – 0.254	172.20.0.255	254	~200
app	172.20.1.0/25	255.255.255.128	172.20.1.1 – 1.126	172.20.1.127	126	~100
db	172.20.1.128/26	255.255.255.192	172.20.1.129 – 1.190	172.20.1.191	62	~20
резерв	172.20.1.192/26 + 172.20.2.0/23	—	—	—	62 + 510	ріст / друга AZ

Логіка безпеки, яку дає така нарізка: security-групи/ACL пишуться на рівні підмереж — «в db-tier можна лише з 172.20.1.0/25 на порт 5432», web дивиться в інтернет, db не має публічного маршруту взагалі. Примітка для реального AWS: у кожній сабнеті AWS резервує 5 адрес (.0 — мережа, .1 — роутер, .2 — DNS, .3 — запас, остання — broadcast), тож у /24 фактично 251 хост — на вимоги плану це не впливає.

6. Опціонально: conntrack -L під час завантаження сайту

На macOS conntrack немає, тому демонстрація в Docker Desktop: контейнер `dz5-curler` (172.17.0.2) у циклі завантажує example.com, а таблицю станів дивимось із host-netns Linux-VM:

```
$ conntrack -L | grep "dport=80 " | grep 172.17.0.2
tcp 6 119 TIME_WAIT src=172.17.0.2 dst=172.66.147.243 sport=58756 dport=80
      src=172.66.147.243 dst=192.168.65.3 sport=80 dport=58756 [ASSURED]
```

Що це означає: це і є мій запис трансляції. Перша половина рядка — оригінальний напрямок (контейнер 172.17.0.2 → example.com:80), друга — очікувана відповідь, і в ній видно NAT: сервер відповідає не контейнеру, а на **192.168.65.3** — адресу VM після MASQUERADE. Conntrack пам'ятає цю відповідність і переписує адреси у зворотних пакетах. `TIME_WAIT` — стан після закриття TCP-сесії, `[ASSURED]` — обмін пройшов в обидва боки, запис захищений від витіснення. Всього в таблиці було 1480 записів. Це той самий механізм, завдяки якому мій домашній роутер знає, кому з 10.20.50.0/24 віддати зворотний пакет. Причому в моєму випадку NAT подвійний: роутер транслює мене у свою WAN-адресу, а публічну 91.226.253.120 навішує вже CGNAT провайдера — саме тому в traceroute з ДЗ 4 хопи 2–3 були приватними адресами (10.12.18.1, 10.1.1.3), і кожен із цих NAT-ів веде власну таблицю станів, як та, що вище. Повний вивід: <docs/conntrack-output.txt>.

7. Опціонально: dig AAAA + curl -6 проти А-запису

```
$ dig AAAA google.com +noall +answer
google.com. 43 IN AAAA 2a00:1450:4001:c0f::8a (та ще 3 адреси ::64/::65/::66)

$ dig A google.com +noall +answer
google.com. 300 IN A 142.251.110.113 (та ще 5 адрес .100–.139)
```

Порівняння: А-записи — шість 32-бітних адрес; AAAA — чотири 128-бітні, всі в одній /64-мережі дата-центру Google (:: у записі стискає нульові групи). Обидва запити обробив той самий локальний DNS 192.168.50.2 за ~50 мс — **сам запит AAAA їде по IPv4-транспорті**, IPv6-зв'язність для цього не потрібна.

А от скористатися відповіддю не вийде: `curl -6 google.com` замість реальних AAAA отримує IPv4-mapped `::ffff:142.251.110.100` (система знає, що v6-маршруту нема), а чесне з'єднання з `2a00:1450:4001:c0f::64` падає з «No route to host». Головний висновок: **DNS-рівень IPv6 у мене працює, транспортний — ні**; це незалежні речі. Повні виводи: <docs/dig-aaaa-output.txt>.